Language modeling

Tokens, cross-entropy, and the numbers you will stare at for months

Start a pre-training run on a model with GPT-2's vocabulary and look at the very first loss it prints: about 10.8. Not 100, not 3.7 — 10.8, every time, on any dataset, before the model has learned a single thing. By the end of this chapter you will know exactly where that number comes from (it is $\ln 50{,}257$, and the 50,257 is a design decision someone made in 2019), why the first day of training is a cliff and the next three months are a grind, why a difference of 0.05 in that number is worth real money, and what it means — concretely, mechanically — when we say the model has "learned language." Chapter 1 built the training recipe with a placeholder loss; this chapter installs the real one, and it is the loss you will stare at for the rest of the book.

The plan follows the data. Text has to become integers before a model can touch it (tokenization), integers have to become vectors (embeddings), the model has to be given a job precise enough to compute a gradient from (next-token prediction, scored by cross-entropy), and its output — a probability distribution — has to be turned back into text (decoding). Each of those four transformations is an efficiency decision in disguise, and two of them, tokenization and the loss, will follow us through the entire book.

2.1 From text to tokens to integers

A neural network is arithmetic; it eats numbers. Text is a stream of bytes. Something must chop the stream into units and assign each unit an integer id, and that something — the *tokenizer* — is decided before training starts and frozen forever after. It is the least glamorous component in the stack and one of the most consequential, because the model will never see text; it will only ever see the tokenizer's output, and every weakness of the chopping is inherited by everything downstream.

The two obvious chopping schemes both fail at scale, and it is worth seeing why. Chop into *characters* and the vocabulary is tiny and nothing is ever out of vocabulary — but a 300-word paragraph becomes ~1,500 units, and (Chapter 3 will make this quantitative) the cost of attention grows quadratically with sequence length, so you are paying roughly 25× the compute of a scheme with 5× fewer units, and asking the model to rediscover from scratch that t-h-e is a word. Chop into *words* and sequences are short — but the vocabulary is unbounded ("unfriendliness", "COVID-19", "ftw", every typo ever made), and any word not in the list at training time becomes a useless `<UNK>` at inference time. Real systems live between the two: frequent strings get their own token, rare strings are built from pieces, and *nothing* is ever unknown because in the worst case a string falls back to raw bytes.

The algorithm that owns this middle ground is byte-pair encoding, BPE — originally a 1994 compression trick,[1] imported into NLP in 2016.[2] It is greedy and almost embarrassingly simple: start with single characters as your vocabulary, count every adjacent pair of units in the corpus, merge the most frequent pair into a new unit, repeat until the vocabulary reaches the size you chose. That is the whole training algorithm. Let us run it for real on a corpus small enough to inspect:

"the faster runner ran past the fastest swimmer the swimmer swam faster"

We trained BPE on that sentence (the code is below; `</w>` is the end-of-word marker, so the algorithm can learn that a word *ends*). The first ten merges, in order, with their frequencies: `st` (5), `er` (5), `er</w>` (5), `ast` (4), `th` (3), `the` (3), `the</w>` (3), `fast` (3), `sw` (3), `faster</w>` (2). Look at what the statistics discovered without being told anything about English: the comparative suffix *-er*, the superlative building block *-st*, the word *the* as a single unit, the stem *fast*. After ten merges, the corpus segments like this:

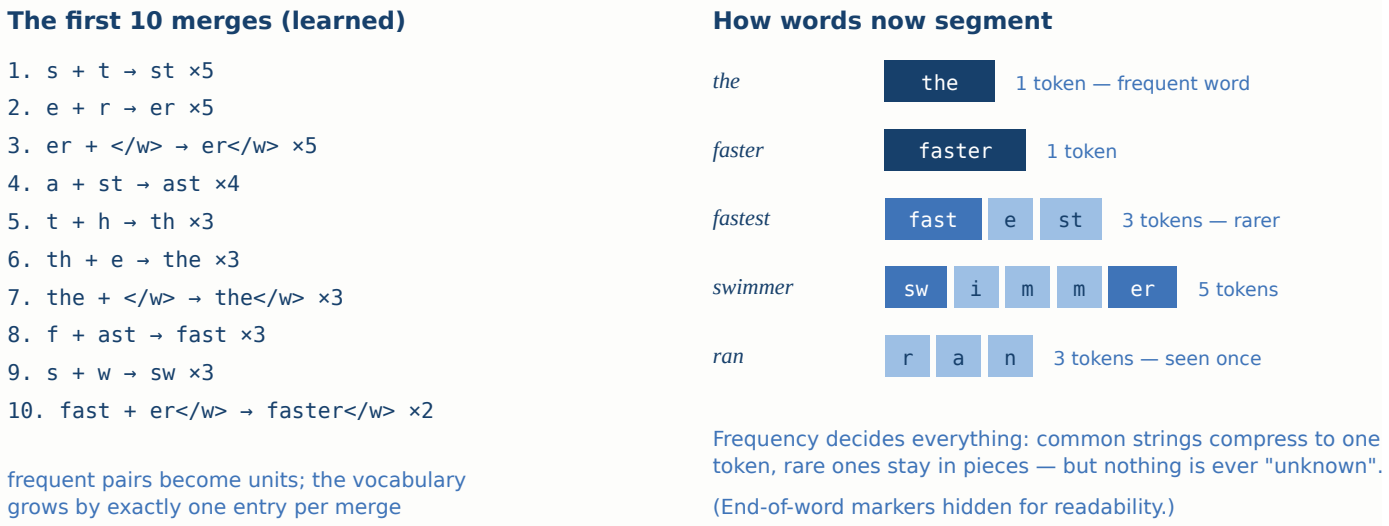

Figure 2.1 — BPE, run for real on a twelve-word corpus. Left: the ten merges the algorithm chose, purely by pair frequency — it found the suffix *-er*, the stem *fast*, and the word *the* with no linguistic knowledge at all. Right: the resulting segmentations. *faster* (frequent) became one token; *fastest* (rarer) is spelled from three pieces. Production tokenizers are this exact algorithm run on terabytes with a vocabulary budget of 32,000–128,000.

Here is the trainer, complete. It is short enough that we would rather show it than describe it — and short enough that you can rerun the experiment above yourself:

```
from collections import Counter

words = Counter("the faster runner ran past the fastest swimmer "
                "the swimmer swam faster".split())
vocab = {w: list(w) + ["</w>"] for w in words}       # start: characters

def pair_counts():
    c = Counter()
    for w, n in words.items():
        for a, b in zip(vocab[w], vocab[w][1:]):
```

```
            c[(a, b)] += n
    return c

for step in range(10):
    (a, b), n = pair_counts().most_common(1)[0]       # most frequent pair
    for w in vocab:                                   # merge it everywhere
        s, out, i = vocab[w], [], 0
        while i < len(s):
            if i < len(s)-1 and (s[i], s[i+1]) == (a, b):
                out.append(a + b); i += 2
            else:
                out.append(s[i]); i += 1
        vocab[w] = out
    print(step + 1, a + b, n)                         # the table in Fig. 2.1
```

Production tokenizers differ from this in engineering, not in idea. GPT-2 runs BPE over *bytes* rather than characters, so the base alphabet is 256 symbols and literally any input — emoji, Korean, binary garbage — is tokenizable; its vocabulary is 50,257 entries: 256 bytes, 50,000 learned merges, and one special end-of-text token.[3] Llama 2 uses the same family of algorithm with 32,000 entries;[4] Llama 3 grew the budget to 128,256 because a bigger vocabulary compresses text into fewer tokens. And with that word — *fewer* — tokenization stops being a preprocessing detail and becomes a line item in your budget. Every training document, every user prompt, every generated reply is billed, in compute and in memory, per token. A tokenizer trained mostly on English spells English efficiently and everything else expensively; the same sentence can cost 30–50% more tokens in Spanish than in English through an English-centric tokenizer, which means 30–50% more compute for the same meaning. We will quantify that tax, and how to avoid paying it, in Chapter 11; for now, remember that "how long is my text" is measured in tokens, the tokenizer decides what a token is, and the decision was frozen before step one of training.

2.2 Embeddings: integers become vectors

Token ids are the wrong kind of number. Id 3797 (`·cat` in GPT-2's vocabulary) is not "more" than id 464 (`The`); the integers are arbitrary labels, and feeding them into arithmetic as magnitudes would teach the model nonsense. What the model actually consumes is one learned vector per vocabulary entry, stored in a matrix with V rows and d columns; tokenizing gives ids, and the ids just select rows. Chapter 3 opened with this picture (its Figure 3.1), so here we only add the part that chapter deferred: where the vectors come from, and why their geometry ends up meaningful.

They come from nowhere special: the embedding matrix is initialized with random noise and updated by gradient descent like every other parameter, following Chapter 1's recipe with no modification — the lookup is differentiable (the gradient for a row flows to exactly the tokens that used it), so rows drift wherever prediction accuracy pushes them. The interesting part is where they drift. Two tokens that occur in interchangeable contexts — `·cat` and `·dog` , `·Tuesday` and `·Wednesday` — receive nearly interchangeable gradients, so their vectors converge toward the same neighborhood; tokens with nothing in common get pushed apart. Similarity of *usage* becomes proximity in the vector space, without anyone defining similarity. This effect was the founding observation of neural language modeling[5] and became famous when simple arithmetic on such vectors — *king* − *man* + *woman* ≈ *queen* — was shown to work embarrassingly often.[6] In a modern LLM the embedding layer is just the ground floor (the residual stream

refines the representation at every layer), but the principle scales up unchanged: meaning, for a model, *is* position in a learned vector space.

Two engineering notes before we move on, both of which return later. Size: the embedding matrix is $V \times d$, so at Llama 3's $V = 128{,}256$ and $d = 4096$ it holds 525 million parameters, and the output layer that maps the final hidden state back to vocabulary logits is another matrix of the same shape — Chapter 3 counts both, and Chapter 11 weighs them against the compression a big vocabulary buys. And symmetry: that output matrix is doing the inverse job of the input one (vectors → tokens instead of tokens → vectors), which is why small models often *tie* the two into a single shared matrix, trading a little quality for Vd parameters — the `tie_word_embeddings` flag you met in Chapter 3's config-reading guide.

2.3 Next-token prediction and cross-entropy

Chapter 1 said you write the loss and the loss writes the program. For a language model, then, everything hangs on one question: what is the loss? The field's answer is almost provocatively simple. Take any text. Hide the next token. Make the model assign a probability to every token in the vocabulary. Score it by the probability it assigned to the token that actually came next. That's it — that is the entire objective behind every model in this book, and one of this book's recurring themes is how much that simplicity is worth operationally: no labels to buy, no annotators to manage; every scrap of text ever written is born pre-labeled, because the "label" of a prefix is simply the token that follows it.

Make it mechanical. The model's raw output at a position is a vector of V real numbers, the *logits* z — one score per vocabulary entry, unnormalized, any sign. The softmax turns scores into a probability distribution:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}},$$

positive, summing to one, order-preserving — the biggest logit becomes the biggest probability. And the loss at that position is the log of one single entry of p: the one belonging to the true next token t:

$$L = -\log p_t.$$

This is *cross-entropy*, and its shape is worth feeling with numbers. Put probability 0.7 on the right token and you pay $-\ln 0.7 = 0.36$. Spread probability uniformly over five candidates and you pay $-\ln 0.2 = 1.61$. Put only 0.02 on the right answer and you pay 3.91 — and the bill keeps growing without bound as your probability approaches zero, which is the loss telling you that *confident wrongness is the worst sin*: being sure of the wrong token costs far more than being honestly uncertain. Gradient descent, always pushing this number down, therefore learns not just to rank the right token first but to place calibrated bets across the whole vocabulary. And now the opening mystery dissolves: an untrained model, all weights noise, spreads probability roughly uniformly over the vocabulary, paying $-\ln(1/V)$ — for $V = 50{,}257$, exactly the 10.8 on your screen at step zero.

The last piece is what makes the objective so compute-efficient, and Figure 2.2 draws it. You do not train on one hidden token per sequence. The causal mask from Chapter 3 means the model's output at *every* position is a prediction about that position's successor, made without seeing it — so one forward pass over an L-token sequence yields L

simultaneous predictions, L cross-entropy terms, L gradients. The training loss is their average, and in code the whole trick is one shift: the labels are the input ids, moved left by one.

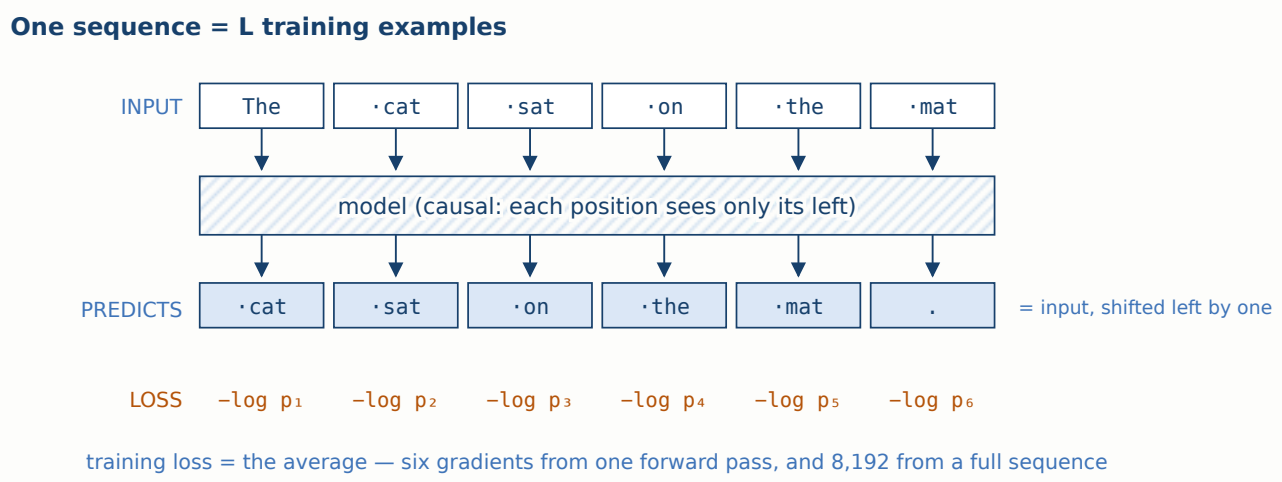

Figure 2.2 — Why next-token prediction is compute-efficient. The causal mask makes every position's output a genuine prediction (it never saw its target), so a single forward-backward pass over the sequence trains on all positions at once. No labels were created; the shifted input *is* the label. This density of signal per FLOP is a large part of why decoder-only models won (Chapter 3.5).

In PyTorch the whole objective is three lines, and the shift is the part people get wrong (Section 2.7):

```
import torch.nn.functional as F

logits = model(ids)                       # (B, L, V)
loss = F.cross_entropy(
    logits[:, :-1].reshape(-1, V),        # predictions at positions 0..L-2
    ids[:,  1:].reshape(-1),              # targets:   positions 1..L-1
)
loss.backward()                           # Chapter 1 takes it from here
```

One vocabulary of terminology and we are done: this loss is measured in *nats* (natural log); divide by $\ln 2$ and it is *bits per token*, which connects it to information theory — a model with loss 2.77 nats ≈ 4 bits per token is, quite literally, a compressor that can encode its training distribution at 4 bits per token. That equivalence between predicting and compressing is not a metaphor, and Section 2.6 leans on it.

2.4 Perplexity, and how to read a loss curve

The community quotes the same quantity in a second form. *Perplexity* is the exponential of the cross-entropy loss,

$$\mathrm{PPL} = e^{L},$$

and its virtue is that it has units you can feel: a perplexity of 20 means the model is, on average, as uncertain as if it were choosing uniformly among 20 equally likely tokens. An effective branching factor. The untrained model's perplexity is the full vocabulary — 50,257, nowhere to go but down — and every drop in loss shrinks the effective menu:

Table 2.1 — The loss ↔ perplexity dictionary. Landmarks worth memorizing; the loss column is what your logs print, the perplexity column is what it feels like. For calibration: GPT-2's 1.5B model, zero-shot, measured 18.3 perplexity on the WikiText-2 benchmark in 2019[3] — a bar that a well-trained model a hundred times smaller clears today.

LOSS (NATS/TOKEN)	PERPLEXITY	WHAT IT MEANS
10.82	50,257	Step zero with GPT-2's vocabulary: uniform guessing.
6.90	~1,000	Minutes in: unigram statistics — frequent tokens are frequent.
4.00	54.6	Local grammar; short-range patterns.
3.00	20.1	A competent small model on clean English text.
2.30	10.0	Strong pre-trained models on web text: one of ten.
1.70	5.5	Frontier territory on held-out data — and near the wall below.

Why is there a wall? Because language itself is uncertain. After "I'll meet you at the", many continuations are legitimately possible; no model, however large, can know which one the author chose, and that irreducible uncertainty puts a floor under the loss that Shannon estimated for English back in 1951 by having humans play exactly this next-symbol guessing game.[7] Loss zero is not the goal and never was; the goal is the floor, and the distance to it shrinks slowly and expensively.

Which brings us to the shape of the curve you will actually watch. It is a cliff followed by a grind, and both phases have a reason. The cliff — 10.8 down to 4-ish, often within the first fraction of a percent of training — is the model harvesting cheap statistics: token frequencies first (that alone buys the drop to ~7), then bigram-ish patterns, then local grammar; each is common in the data, so its gradient signal is loud and learning is fast. The grind is everything after: the remaining loss lives in the *tail* — rare facts, long-range dependencies, reasoning — where each phenomenon appears rarely and buys a sliver of loss. That long flat stretch is where all the money goes, and it is why loss deltas that look laughable are treated with reverence: between two multi-billion-token runs, a difference of 0.02 nats on held-out data is a real, reproducible gap in capability — the smooth, predictable relationship between compute spent and loss achieved[8] is the entire subject of Chapter 7's scaling laws, and held-out loss is the book's default health metric (evaluation grows beyond it in Chapter 9).

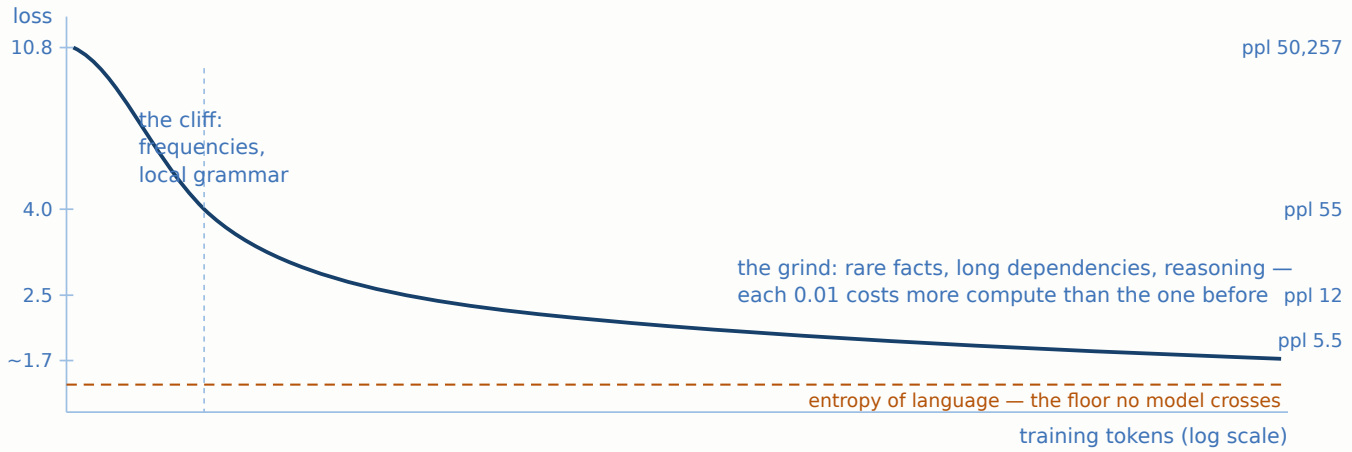

Figure 2.3 — The two-phase shape of every pre-training loss curve (schematic; the axis is log-tokens, which is what makes the grind visible at all). The cliff is cheap statistics with loud gradients; the grind is the long tail of language, where the loss earns its reputation as the most expensive number in computing. The dashed floor is the irreducible entropy of text itself.[7]

2.5 Decoding: greedy, sampling, temperature

Everything so far trains the model. Using it raises a question training never had to answer. The model's output is a distribution over 50,257 tokens; a chatbot must emit exactly one. The rule that turns distributions into tokens is the *decoding strategy*, it runs in a loop (pick a token, append it, predict again — the autoregressive loop whose memory cost is Chapter 3's KV cache), and it is not part of the model: the same weights write differently under different rules.

The obvious rule is the worst one, which is a genuinely counterintuitive fact worth internalizing. *Greedy* decoding takes the argmax at every step — and produces text that is flat, and, at its characteristic worst, degenerates into loops, repeating a phrase over and over while a human's continuation would wander off into the distribution's tail. Measured against real text, greedy output is *too probable*: humans do not emit maximum-likelihood language.[9] So production systems *sample* from the distribution instead, and control how adventurously with a single knob applied to the logits before the softmax — the *temperature*:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

Divide logits by T < 1 and gaps between them widen, sharpening the distribution toward the favorite; T > 1 flattens it toward uniform. Concretely — computed, not sketched — take five candidate tokens with logits $(5.0,\ 3.0,\ 2.0,\ 1.0,\ 0.5)$:

Table 2.2 — One distribution, three temperatures. Same model output, three personalities. As $T \to 0$, sampling becomes greedy; high temperature hands real probability to the tail, which reads as creativity right up until it reads as nonsense.

CANDIDATE LOGITS (5.0, 3.0, 2.0, 1.0, 0.5)	P_1	P_2	P_3	P_4	P_5

CANDIDATE LOGITS (5.0, 3.0, 2.0, 1.0, 0.5)	P_1	P_2	P_3	P_4	P_5
T = 0.5 — sharpened, near-greedy	0.979	0.018	0.002	0.000	0.000
T = 1.0 — the distribution as trained	0.823	0.111	0.041	0.015	0.009
T = 2.0 — flattened, adventurous	0.546	0.201	0.122	0.074	0.058

Raw sampling has its own failure — once in a while it picks a genuinely terrible token from the far tail, and one bad token derails everything after it — so practical systems truncate the tail before sampling: *top-k* keeps the k most probable tokens; *top-p* (nucleus sampling) keeps the smallest set whose probabilities sum to p, adapting the cutoff to how peaked the distribution is at each step, which is why it became the default.[9] The whole loop fits in a dozen lines:

```
def generate(model, ids, n_new, T=0.8, top_k=50):
    for _ in range(n_new):
        z = model(ids)[:, -1]                 # logits for the next token
        z = z / T                             # temperature
        if top_k:
            kth = z.topk(top_k).values[:, -1:]
            z = z.masked_fill(z < kth, -1e4)  # truncate the tail
        p = z.softmax(-1)
        nxt = torch.multinomial(p, 1)         # sample one token
        ids = torch.cat([ids, nxt], dim=1)    # append and repeat
    return ids
```

Connect this back to the loss and one apparent paradox in the field dissolves: training never decodes. The loss touches the full distribution at every position; sampling strategies exist only at inference. This division of labor is why one set of weights can serve a deterministic code assistant (low temperature) and a brainstorming partner (high temperature) — and why "the model said X" is always, implicitly, "the model plus a decoding rule said X." When Part V trains models on their own generations (rejection sampling in Chapter 25, reinforcement learning in Chapter 24), the decoding rule stops being a serving detail and becomes part of the training loop itself, rollout cost and all.

2.6 What "the model learned" actually means

"It just predicts the next word" is the most common dismissal of language models, and the most common overclaim is its mirror image. Having built the objective from parts, you are now equipped for a more precise position than either.

Start from the compression equivalence of Section 2.3: a model with low cross-entropy on held-out text is, mathematically, a good compressor of that text. Now ask what a compressor of human text must contain. To keep beating the frequency tables it learned in the cliff, it must exploit every regularity that makes text predictable — and pull the thread of "every regularity" and out comes almost everything we call knowledge of language and the world. Spelling and morphology, because *swim* predicts *swimmer*'s pieces (our BPE example found that structure with ten merges and a Counter). Syntax, because after "the keys to the cabinet", the verb is *are*, not *is* — a dependency that jumps the intervening noun. Facts, because "The capital of Peru is" concentrates probability onto one token, and every

fact the model can store shaves loss off every place that fact recurs. Register and style, because a legal filing and a group chat continue differently. Even local reasoning, because text is full of passages whose continuation is *computed* from what precedes them — arithmetic carried out across a sentence, a conclusion following premises — and a predictor that can perform the computation outpredicts one that pattern-matches. None of these were objectives. They are what minimizing one loss on enough text forces into 6.7 billion constants, and the residual stream picture from Chapter 3 is where that knowledge physically sits — attention moving context to where it is needed, MLP key-value pairs firing facts into the stream.

What the objective does *not* grant is just as visible from the construction. The model learned the distribution of text, not the world behind it: where text is systematically wrong or systematically silent, the model inherits the defect at full confidence, and when its distribution is flat it still emits a token — fluent, well-formed, and unmoored, which is all a "hallucination" is. Nothing in $-\log p_t$ rewards saying "I don't know," and nothing in it distinguishes Peruvian Spanish from a Madrid-inflected average if the training data under-represents the former — which is why Part IV of this book exists: data curation decides *which* distribution gets learned (Chapter 10), and post-training reshapes a raw text-continuer into something that answers questions, refuses appropriately, and knows its limits (Part V). The one-sentence version to carry forward: **pre-training buys capability; what the capability is aimed at is decided elsewhere** — and the rest of this book is, in large part, the economics of both.

2.7 Practical considerations

Language modeling's bugs cluster in the seams between text and tensors. These are the ones that cost real debugging days.

The tokenizer and the model are one artifact

Token ids have no meaning outside the tokenizer that produced them: id 3797 is `·cat` in GPT-2's vocabulary and something unrelated in Llama's. Load a checkpoint with the wrong tokenizer and nothing crashes — shapes match as long as the vocab sizes do — the model simply reads gibberish and writes gibberish. Ship them together, version them together, and treat "which tokenizer?" as the first question in any debugging session involving strange outputs. The subtle version of this bug is *drift*: a tokenizer regenerated with a different version of the library, or with different normalization settings, silently re-segments some strings, and a fine-tune on those ids quietly trains against shifted inputs.

Whitespace is part of the token

BPE tokenizers fold the leading space into the word: `cat` and `·cat` are different ids with different embeddings, learned from different contexts. Consequences bite in practice. A prompt ending in a trailing space ("The answer is ") forces the model off its natural segmentation — it now must continue with a *space-less* word token it rarely saw after a space was already emitted — and quality drops for no visible reason. Case matters the same way (`·Cat` is a third id), numbers fragment inconsistently (`2024` one token, `20 24` two — one reason arithmetic is hard for LLMs), and counting "words" by counting tokens is off by 30% before you start. When output looks inexplicably worse, print the actual token ids of your prompt; the answer is usually there.

Special tokens are load-bearing

A handful of reserved ids — beginning-of-sequence, end-of-sequence, padding, and later the chat-template markers of Chapter 23 — are how the model knows where documents start and stop. They are not decoration. EOS is a *trained prediction target*: it is how a model learns that texts end, and how generation knows when to stop; forget to append it between documents and the model learns that text never terminates (and Chapter 16 shows the same marker doing boundary duty inside packed sequences). Padding must be excluded from the loss (the `ignore_index=-100` convention) or the model spends capacity learning to predict padding — a bug that *improves* the printed loss while damaging the model, which makes it a perfect specimen of Chapter 1's lesson that the loss is a proxy.

Common pitfalls

The off-by-one on labels.

The target at position i is token $i{+}1$. Train with unshifted labels and the model learns the identity function — loss plummets toward zero, generation emits the prompt's last token forever. A suspiciously wonderful loss curve on day one is this bug until proven otherwise.

Averaging loss over padding.

Batches padded to equal length must mask pad positions out of the mean; otherwise short-sequence batches report fake-low loss and gradients are diluted. Symptom: loss that jumps whenever batch composition changes.

Softmax over 50K entries in float16.

The logits' exponentials overflow half precision easily; compute the softmax/cross-entropy in float32 (every framework's fused loss does) rather than hand-rolling it in the model's dtype. Chapter 1's "subtract the max" is necessary but not sufficient at vocabulary scale.

Comparing perplexities across tokenizers.

Perplexity is per-*token*; a tokenizer that cuts text into more, easier pieces scores flattering perplexity on the same text. Cross-model comparisons are only valid at matched tokenization — or normalized per byte or per word. This bites hardest in multilingual evaluation, where fertility differs by 50% (Chapter 11).

Judging a model by one greedy sample.

Decoding strategy changes outputs more than many training interventions do. Fix temperature, top-p, and seed before attributing a quality change to your fine-tune — and evaluate on the distribution (held-out loss) as well as on samples.

Trailing whitespace and template mismatches in prompts.

The classic invisible bug in SFT and evaluation both: the training template and the inference template differ by one space or newline, every prompt lands off-distribution, scores drop a few points, nothing errors. Diff the token ids, not the strings.

2.8 Where this shows up later

Chapter 3 already consumed this chapter's output: the embedding lookup that opens it is Section 2.2, and the causal mask that makes Figure 2.2's shifted-label trick sound is its Section 3.2. The loss built here is the book's spine from now on: Chapter 7's scaling laws are curves *of this loss* against compute; Chapter 9 builds evaluation up from held-out loss to benchmarks precisely because per-token loss, powerful as it is, saturates as a progress signal; and Chapter 19's continued pre-training reads its health off this loss's re-warming bump.

Tokenization returns twice with money attached: Chapter 11 turns Section 2.1's closing remark — the 30–50% fertility tax on Spanish and Portuguese through English-centric tokenizers — into measurements and a design procedure, and Chapter 16 packs multiple documents into one sequence, where the EOS tokens and loss masks of Section 2.7 become correctness-critical. Decoding returns transformed: what is a serving preference in Section 2.5 becomes a training cost center when Part V samples from the model inside the loop — rejection sampling for synthetic data (Chapter 25) and RL rollouts (Chapter 24), where Chapter 3's KV cache sets the price. And the limits catalogued in Section 2.6 are the assignment sheet for Chapters 10, 26, and 27: choose the distribution, calibrate the refusals, and check what actually got learned.

2.9 Summary

A language model is Chapter 1's recipe with one loss installed: predict the next token, pay $-\log p$ of the truth. Text reaches the model through a tokenizer — BPE, a greedy frequency-merger that spells common strings in one token and rare ones in pieces, fixed before training and consequential forever — and enters as learned embedding vectors whose geometry comes to encode similarity of usage. The objective needs no labels, trains every position of every sequence at once, and its exponential, perplexity, reads as an effective branching factor: 50,257 at step zero, cliff to double digits on cheap statistics, then a long expensive grind toward the entropy floor of language itself, where differences of 0.02 nats are worth real money. The model outputs distributions; decoding — temperature, top-k, top-p — turns them into text and is not part of the model. And "learned" means exactly this: whatever regularities of text survive into lower held-out loss — grammar, facts, style, some computation — with the objective's blind spots (confabulation, inherited bias, no notion of "I don't know") defining the work of the chapters ahead.

References

1. Gage, P. (1994). A New Algorithm for Data Compression. *The C Users Journal*, 12(2), 23–38. The original byte-pair encoding, proposed as a compression scheme.
2. Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*. arXiv:1508.07909. Imports BPE into NLP as the answer to open vocabularies.
3. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. Byte-level BPE with a 50,257-entry vocabulary; the 1.5B model reports 18.34 perplexity zero-shot on WikiText-2.

4. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. BPE (SentencePiece) tokenizer with a 32,000-token vocabulary; Meta AI (2024), The Llama 3 Herd of Models, arXiv:2407.21783, grows it to 128,256.

5. Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A Neural Probabilistic Language Model. *JMLR*, 3, 1137–1155. Learned word embeddings trained jointly with a next-word objective.

6. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781. The word-analogy arithmetic (*king* − *man* + *woman* ≈ *queen*) on embeddings trained from context prediction.

7. Shannon, C. E. (1951). Prediction and Entropy of Printed English. *Bell System Technical Journal*, 30(1), 50–64. Estimates the entropy of English via human next-symbol prediction — the floor under every loss curve.

8. Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361. Cross-entropy loss falls smoothly and predictably with model size, data, and compute — the empirical basis for treating small loss deltas as meaningful.

9. Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The Curious Case of Neural Text Degeneration. *ICLR*. arXiv:1904.09751. Documents greedy/beam degeneration and repetition, shows human text is not maximum-probability text, and introduces nucleus (top-p) sampling.